



Algoritmos de Ordenação baseados em Estruturas de Dados

Alexandre Dâmaso · Julya Pires · Mariana Kaori Yano · Rita Marie

Engenharia de Computação — CEFET-MG, Divinópolis

Agenda

01

Introdução

Contextualização do problema de ordenação

02

Fundamentação Teórica

Heap binário, d-ário e SmoothSort

03

Implementação

Pseudocódigos e estratégias

04

Experimentos

Ambiente, linguagens e métricas

05

Resultados

Análise comparativa e conclusões



O Problema da Ordenação

Organizar um conjunto de elementos segundo uma relação de ordem — crescente, decrescente ou lexicográfica — é um dos problemas mais estudados da Ciência da Computação.

Definição Formal

Dada uma sequência de **n** elementos:

$$\langle a_1, a_2, \dots, a_n \rangle$$

O objetivo é rearranjar seus elementos obtendo uma sequência ordenada:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

O algoritmo opera exclusivamente sobre as **chaves** dos registros, garantindo eficiência e generalidade.

Por que Ordenação Importa?



Busca Eficiente

Dados ordenados permitem busca binária em **$O(\log n)$**



Otimização de Processos

Escalonamento de tarefas e filas de prioridade



Análise de Dados

Base para algoritmos de junção, agrupamento e relatórios



Restrições de Memória

Algoritmos in-place são essenciais em sistemas embarcados e IoT

CAPÍTULO 1

Algoritmos Estudados

HeapSort Binário, HeapSort D-ário e SmoothSort — três abordagens baseadas na estrutura Heap, com trade-offs distintos entre simplicidade, eficiência e adaptatividade.

HeapSort — Visão Geral

Desenvolvido por **J.W.J. Williams (1964)**, com versão anterior de Floyd (1962) chamada Treesort. Combina ordenação local (como InsertionSort) com uso constante de memória auxiliar (como MergeSort).



Vantagens e Desvantagens do HeapSort

✓ Vantagens

- Complexidade **$O(n \log n)$** garantida em todos os casos
- Execução **in-place** — espaço auxiliar $O(1)$
- Implementação iterativa possível
- Sem degradação no pior caso (ao contrário do QuickSort)

✗ Desvantagens

- Não é estável — não preserva ordem de elementos iguais
- Padrões de acesso não sequenciais → mais falhas de cache
- Na prática, mais lento que QuickSort
- Implementação mais complexa para iniciantes

Aplicações Reais do HeapSort

Sistemas Embarcados / IoT

Natureza in-place permite ordenação com RAM mínima, sem alocação dinâmica

Sistemas de Tempo Real

Comportamento determinístico $O(n \log n)$ — essencial em controles aeronáuticos e robótica

Filas de Prioridade

Estrutura heap usada diretamente em escalonamento de processos em SOs

SmoothSort — A Alternativa Adaptativa

Proposto por **Edsger W. Dijkstra** na década de 1980 para superar a principal limitação do HeapSort: a incapacidade de aproveitar dados parcialmente ordenados.



Números de Leonardo

O SmoothSort utiliza uma sequência numérica criada especificamente para o algoritmo:

$$L(n) = \begin{cases} 1, & \text{se } n = 0 \\ 1, & \text{se } n = 1 \\ L(n-1) + L(n-2) + 1, & \text{se } n > 1 \end{cases}$$

Gerando a sequência: **1, 1, 3, 5, 9, 15, 25, 41, 67, ...**

Essa sequência permite decompor o vetor em uma **floresta de heaps** de tamanhos variados, possibilitando fusões eficientes sem memória auxiliar.

Vantagens e Desvantagens do SmoothSort

✓ Vantagens

- Complexidade **$O(n)$** no melhor caso (entrada ordenada)
- Mantém $O(n \log n)$ no pior caso
- In-place com espaço auxiliar $O(1)$
- Pioneiro em ordenação adaptativa in-place

✗ Desvantagens

- Implementação muito mais complexa
- Gerenciamento de bitmap para floresta de heaps
- Em dados aleatórios, não supera métodos mais simples
- Pouco adotado em bibliotecas padrão

Aplicações Reais do SmoothSort

Logs de Sistemas

Dados organizados cronologicamente — ordenação prévia é aproveitada para reduzir custo a $O(n)$

Fluxos de Sensores

Dados com variações incrementais — estrutura parcial explorada pelo algoritmo adaptativo

Tabelas de Pontuação

Atualizações contínuas em dados quase ordenados — ideal para o perfil adaptativo do SmoothSort

CAPÍTULO 2

Fundamentação Teórica

Estrutura Heap, propriedades, operações fundamentais e generalização d-ária.

A Estrutura Heap

Representação Implícita em Array

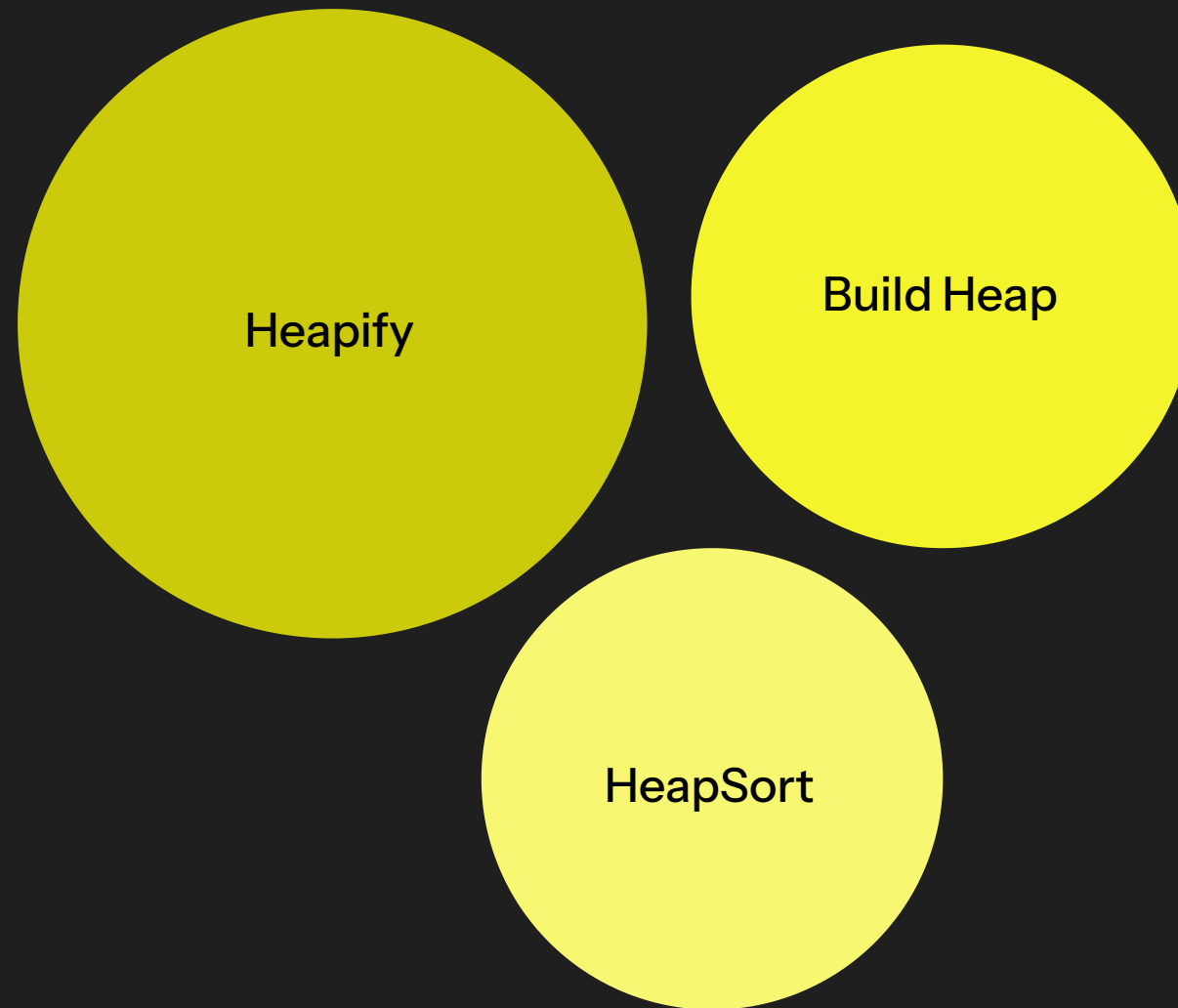
Dado um nó i :

- Filho esquerdo: $2i + 1$
- Filho direito: $2i + 2$
- Pai: $\lfloor (i-1)/2 \rfloor$

Max-heap: pai $\geq$ filhos (máximo na raiz)

Min-heap: pai $\leq$ filhos (mínimo na raiz)

Operações Fundamentais



O **Build Heap** tem custo linear $O(n)$ — análise mais precisa que a estimativa ingênua $O(n \log n)$, pois a maioria dos nós está próxima às folhas.

Heap D-ário — Generalização

Cada nó possui até d filhos

Proposto por Johnson e formalizado por Tarjan. Os filhos do nó i ocupam posições $d \cdot i + 1$ até $d \cdot i + d$.

Tarjan observou que $d = 3$ ou $d = 4$ tendem a superar $d = 2$ em todas as circunstâncias.

Altura reduzida

$\log_d n$ — menos níveis percorridos

Mais comparações/nível

d filhos comparados em cada Heapify

Melhor localidade

Menos falhas de cache em hardware moderno

Comparação Teórica: HeapSort D-ário vs. SmoothSort

Propriedade	HeapSort D-ário	SmoothSort
Melhor caso	$O(n \log_d n)$	$O(n)$
Caso médio	$O(n \log_d n)$	$O(n \log n)$
Pior caso	$O(n \log_d n)$	$O(n \log n)$
Espaço auxiliar	$O(\log_d n)$	$O(1)$
In-place	Sim	Sim
Estável	Não	Não
Adaptativo	Não	Sim

SmoothSort — Estrutura Interna

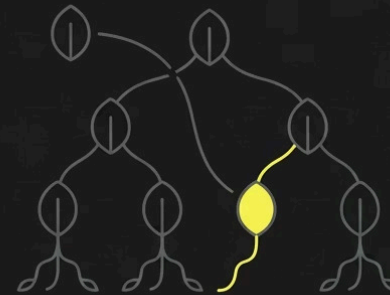
Floresta de Heaps de Leonardo

O vetor é decomposto em árvores cujos tamanhos seguem a sequência de Leonardo. Cada árvore de tamanho L_n possui:

- Filho esquerdo: árvore de tamanho L_{n-1}
- Filho direito: árvore de tamanho L_{n-2}
- Raiz: maior elemento da árvore

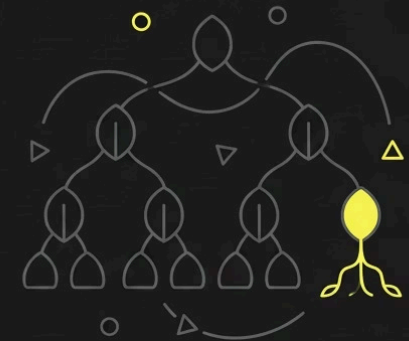
As raízes da floresta são mantidas em **ordem crescente** da esquerda para a direita.

FASE 1: CONSTRUÇÃO



**INSERÇÃO
PROGRESSIVA,
MANUTENÇÃO DE
HEAPS, FUSÃO
DE ÁRVORES.**

FASE 2: EXTRAÇÃO



**REMOÇÃO DA
RAIZ, DIVISÃO EM
SUBÁRVORES,
RESTAURAÇÃO
DA ORDEM.**

CAPÍTULO 3

Pseudocódigos

Estrutura algorítmica das variantes implementadas: Heap Max, Heap Min, Heap D-ário e SmoothSort.

HeapSort Max — Pseudocódigo

Fase 1: Build Heap Máximo

```
Para  $i$  de  $\lfloor n/2 \rfloor - 1$  até 0:  
  HEAPIFY_MAX(A, n, i)
```

Fase 2: Extração

```
Para  $i$  de  $n-1$  até 0:  
  Trocar(A[0], A[i])  
  HEAPIFY_MAX(A, i, 0)
```

Procedimento HEAPIFY_MAX

```
maior = i  
esq =  $2*i$ ; dir =  $2*i+1$   
  
Se esq < n e A[esq] > A[maior]:  
  maior = esq  
Se dir < n e A[dir] > A[maior]:  
  maior = dir  
Se maior  $\neq$  i:  
  Trocar A[i] com A[maior]  
  HEAPIFY_MAX(A, n, maior)
```

HeapSort Min — Pseudocódigo

Fase 1: Build Heap Mínimo

```
Para i de  $\lfloor n/2 \rfloor - 1$  até 0:  
  HEAP_MIN(A, n, i)
```

Fase 2: Extração

```
Para i de n-1 até 1:  
  Trocar A[0] com A[i]  
  HEAP_MIN(A, i, 0)
```

Procedimento HEAP_MIN

```
menor = i  
esq =  $2*i$ ; dir =  $2*i+1$   
  
Se esq < n e A[esq] < A[menor]:  
  menor = esq  
Se dir < n e A[dir] < A[menor]:  
  menor = dir  
Se menor  $\neq$  i:  
  Trocar A[i] com A[menor]  
  HEAP_MIN(A, n, menor)
```

❗ Produz ordenação em sentido inverso — mais usado para fins comparativos.

Heap D-ário — Pseudocódigo

Algoritmo Principal

```
HEAP_SORT_D(A, d):  
  n = tamanho de A  
  
  // Fase 1: Build Heap d-ário  
  Para i de  $\lfloor n/d \rfloor$  até 0:  
    HEAPIFY_D(A, n, i, d)  
  
  // Fase 2: Extração  
  Para i de n-1 até 1:  
    Trocar A[0] com A[i]  
    HEAPIFY_D(A, i, 0, d)
```

Diferença-chave

O Heapify percorre **d filhos** por nível em vez de 2, reduzindo a altura da árvore para **$\log_d n$** .

📌 Aridade d=4 mostrou-se o melhor equilíbrio nos experimentos.

SmoothSort — Pseudocódigo

Algoritmo Smoothsort(A):

// Fase 1: Construção da floresta de heaps de Leonardo

Para $i = 0$ até $n-1$:

 Inserir elemento na estrutura de heaps

 Manter propriedade de heap e ordem das raízes

// Fase 2: Extração ordenada

Para $i = n-1$ até 0 :

 Remover raiz da árvore mais à direita

 Se árvore tinha filhos → duas novas árvores independentes

 Restaurar propriedade de heap e ordenação das raízes

Ambas as fases seguem a mesma estrutura: **construção** e **ordenação**. A diferença está na organização dinâmica da floresta de Leonardo.

CAPÍTULO 4

Modelagem Experimental

Ambiente computacional, linguagens utilizadas e definição dos experimentos.

Ambiente Computacional



C + Go

Intel Core i5-12450H, 8 GB RAM, Windows 10 + WSL
Ubuntu, gcc 13.3.0, go1.22.2



C++

Samsung NP300E5M, Intel i5-7200U, 8 GB RAM, g++ 13.3.0
(C++17)



Java

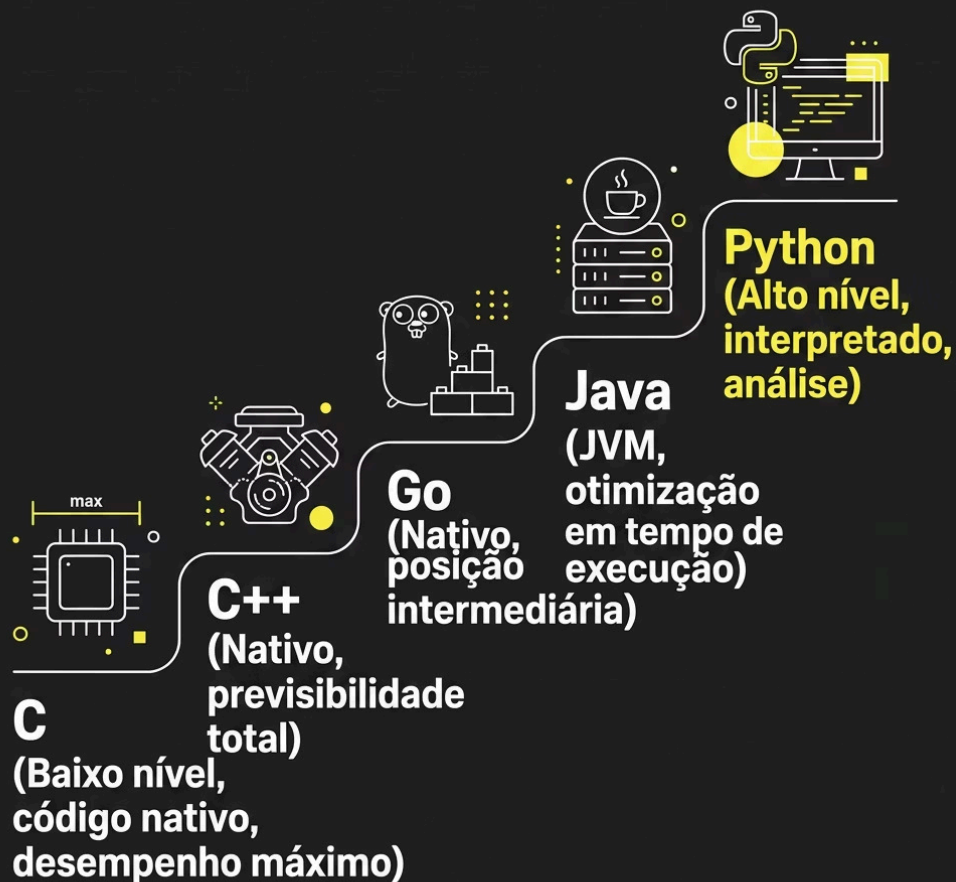
LG Gram 15, Intel i7-8550U, 8 GB RAM, OpenJDK 17.0.15,
Ubuntu 20.04 (WSL)



Python

Acer Nitro, Intel i5-11400H, 8 GB RAM, SSD 512 GB,
Ubuntu 24.04, GTX 1650

Por que Essas Linguagens?



Objetivo da Diversidade

Avaliar o impacto do ambiente de execução nos algoritmos, isolando variáveis como:

- Gerenciamento automático de memória (GC)
- Compilação JIT vs. nativa
- Overhead de abstração

i Python usou Timsort nativo como proxy do SmoothSort — resultados devem ser interpretados como referência do ambiente.

Definição dos Experimentos

Tamanhos de Entrada

$N \in \{10^2, 10^3, 10^4, 10^5, 10^6\}$ elementos

Distribuições

Aleatória · Ordenada crescente · Ordenada decrescente

Aridades D-Heap

$d \in \{2, 3, 4, 8\}$ — $d=2$ equivale ao HeapSort binário clássico

Repetições

C: 3x · C++: 5x · Python: 7x · Java: 20x — médias calculadas

Semente fixa de valor **42** para reprodutibilidade. Vetor original clonado antes de cada repetição.

Métricas Coletadas



Tempo de Execução

Medido em ms via `System.nanoTime()` (Java) e cronômetro de alta resolução (C++/Python)



Consumo de Memória

Java: memória fixa ($N \times 4$ bytes) + dinâmica via Runtime.
C++/Python: tamanho do vetor de entrada



Movimentações

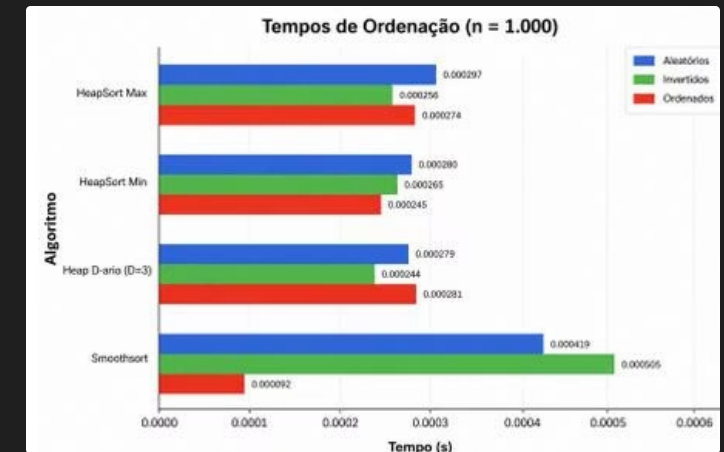
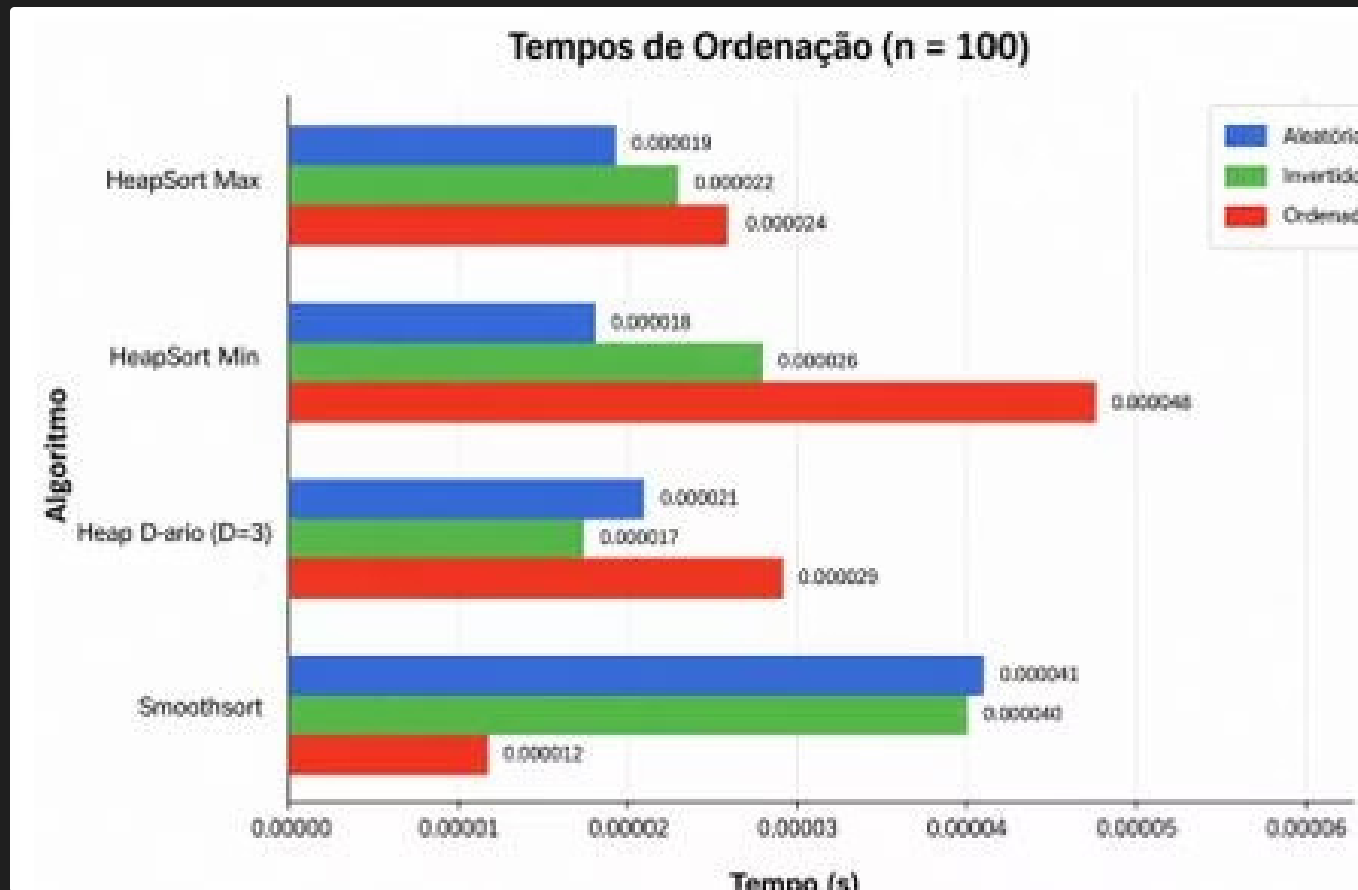
D-Heap: swaps completos.
SmoothSort: atribuições individuais (diferença considerada na análise)

CAPÍTULO 5

Resultados — Linguagem C

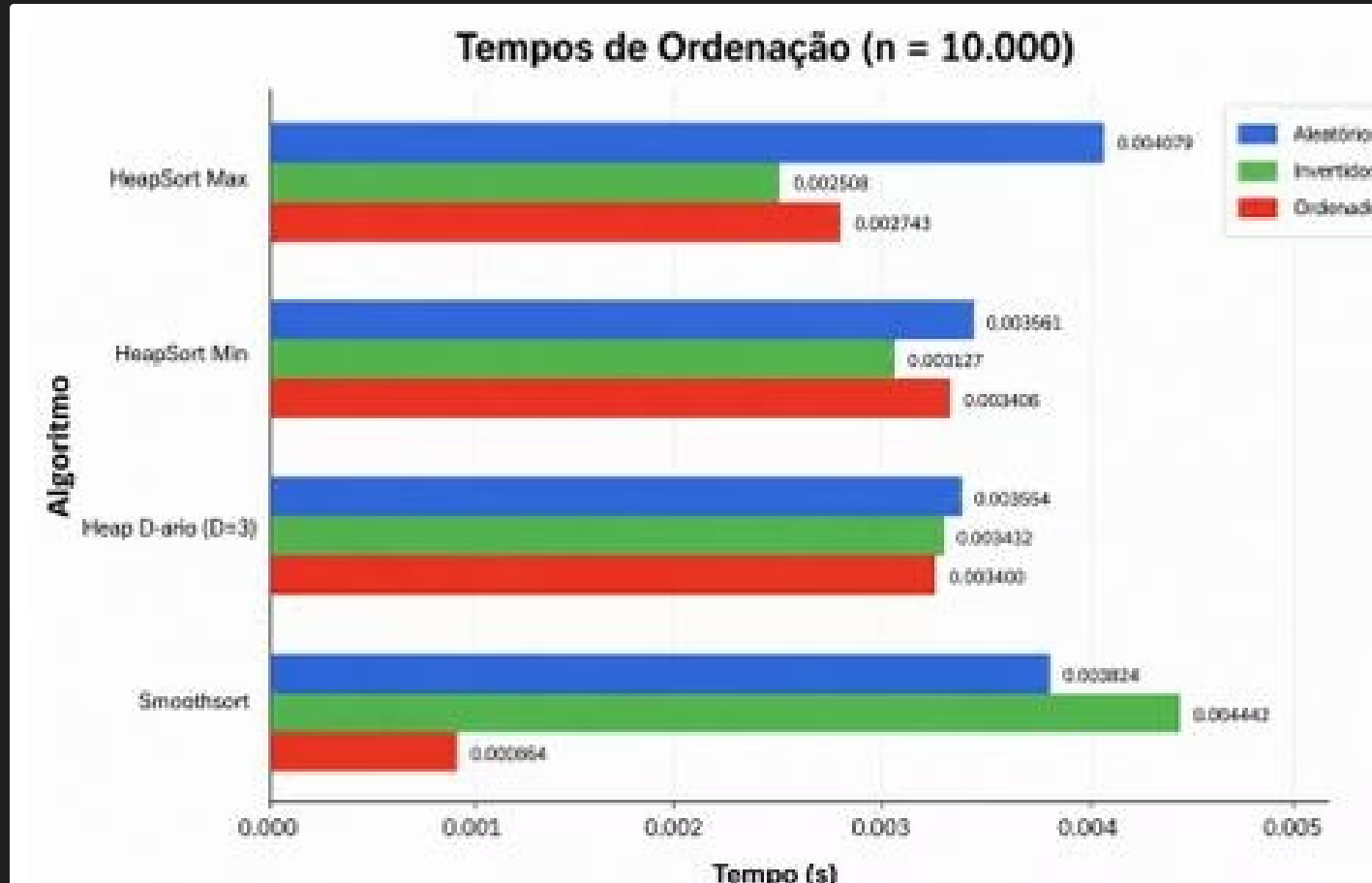
Testes com N até 10^6 , 3 execuções por cenário, média calculada.

C — Resultados: $n = 100$ e $n = 1.000$



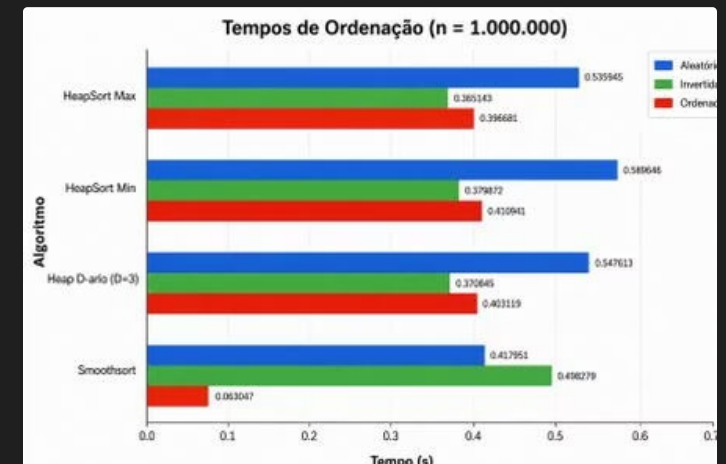
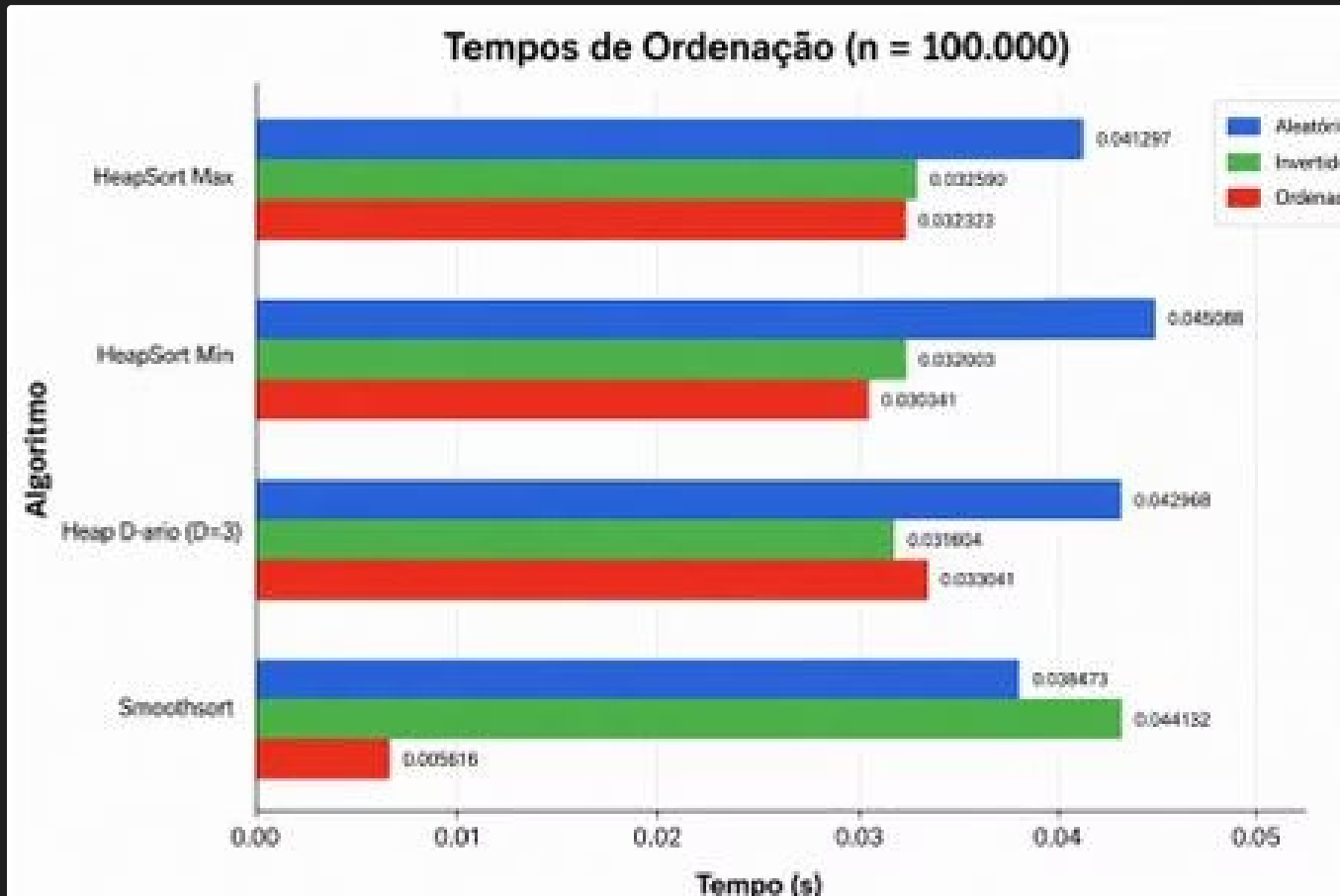
Para $n=100$, o SmoothSort é mais lento em dados aleatórios/invertidos, mas já demonstra vantagem em dados ordenados (0,000012 s vs. ~0,000024 s do HeapSort Max).

C — Resultados: $n = 10.000$



Com $n=10^4$, o SmoothSort atinge **0,000864 s** em dados ordenados — **~5x mais rápido** que os demais. Em dados aleatórios, os algoritmos convergem.

C — Resultados: $n = 100.000$ e $n = 1.000.000$



Em $n=10^6$, SmoothSort ordena dados ordenados em **0,063 s** vs. ~0,45 s dos demais. Em dados aleatórios, **Heap D-ário** lidera com 0,547 s.

C — Análise por Tipo de Entrada

Aleatória

Heap D-ário apresenta melhor desempenho. SmoothSort supera HeapSort Max e Min.

Invertida

Heap D-ário continua liderando. SmoothSort não apresenta vantagem nesse cenário.

Ordenada

SmoothSort domina amplamente — algoritmo adaptativo aproveita a ordem existente.

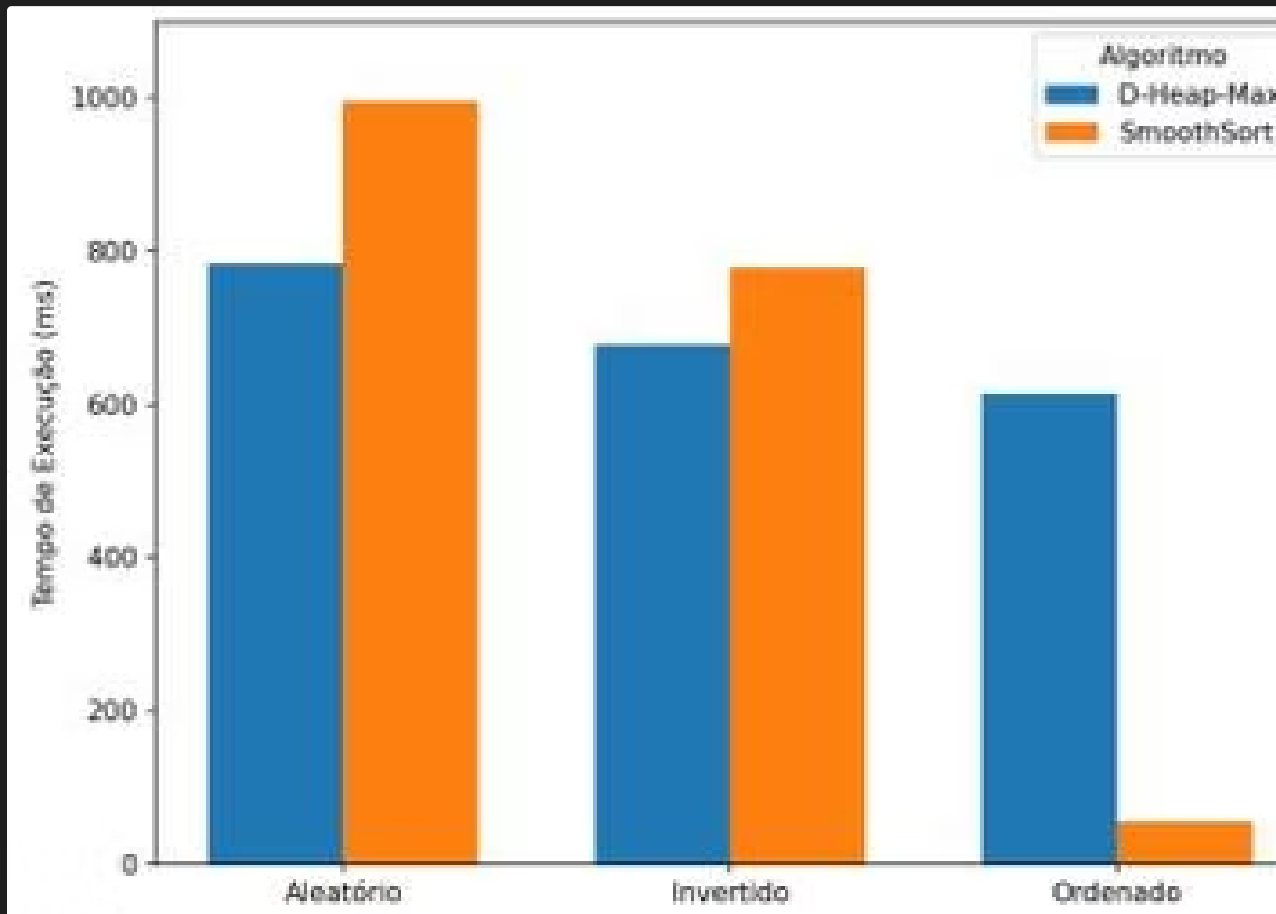
✓ Consumo de memória adicional: 0 KB em todos os casos — confirmando complexidade espacial $O(1)$.

CAPÍTULO 5

Resultados — C++

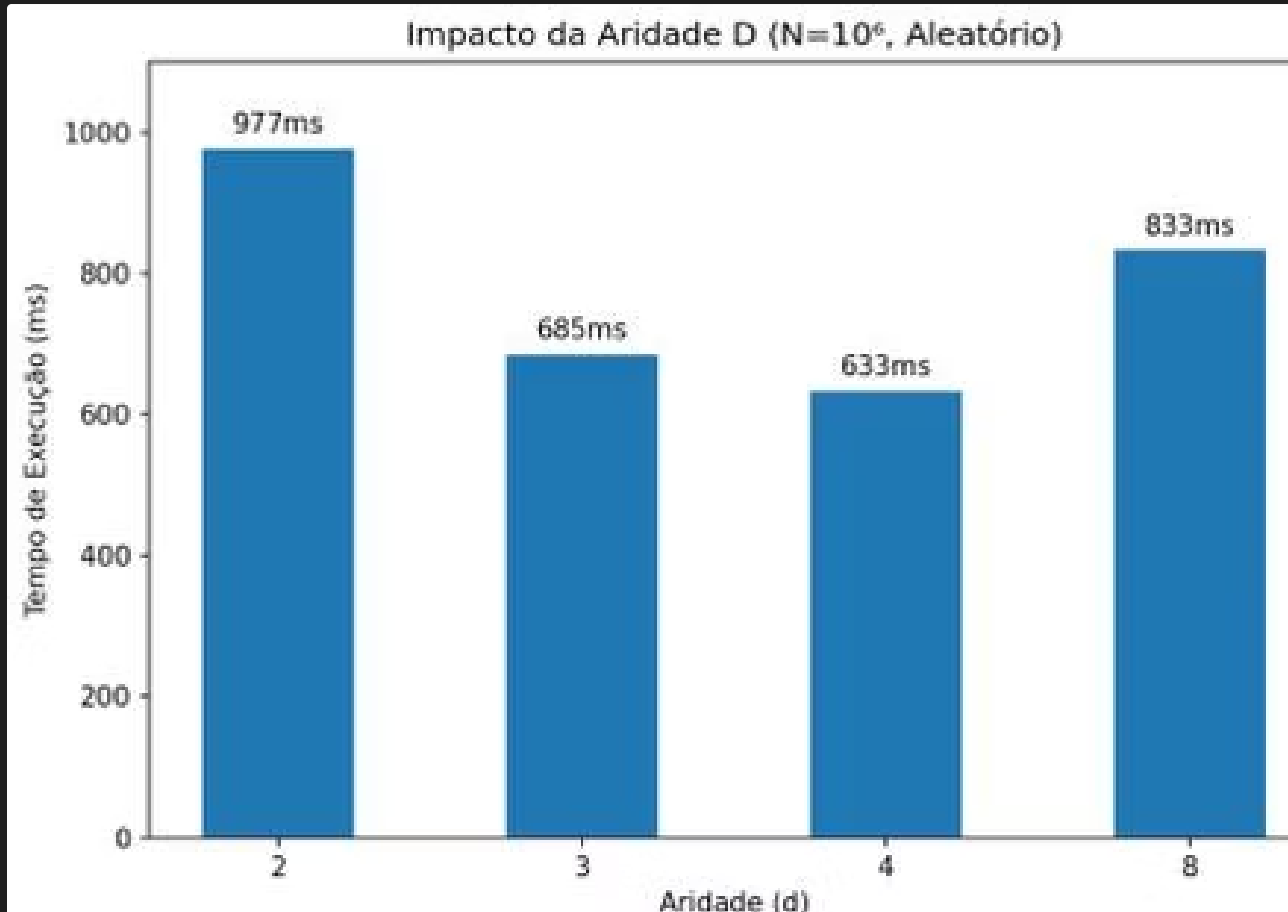
5 execuções por cenário. Aridades $d \in \{2, 3, 4, 8\}$. N de 10^2 a 10^6 .

C++ — Adaptabilidade ($N = 10^6$)



Entrada	D-Heap-Max (ms)	SmoothSort (ms)
Aleatório	~796	~1000
Invertido	~685	~779
Ordenado	~633	~56

C++ — Impacto da Aridade D



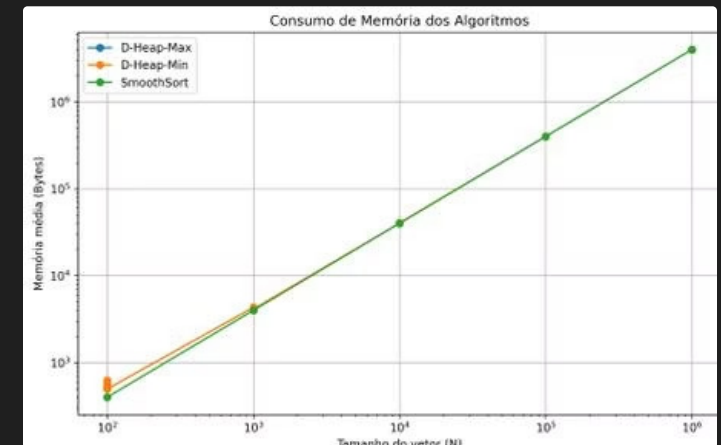
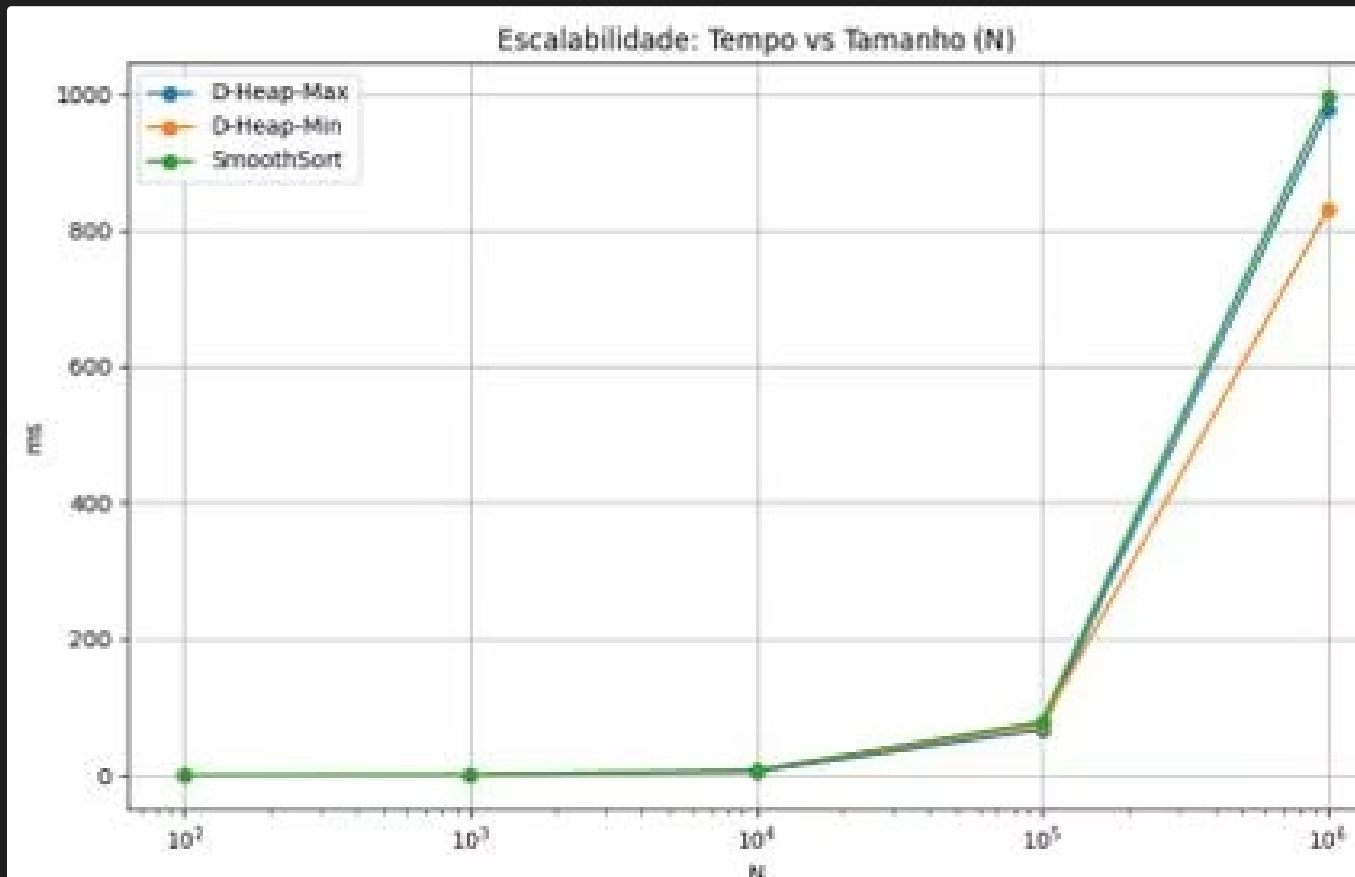
Entre d=2 e d=4: queda de **977 ms** → **633 ms**. Com d=8, o tempo sobe para 833 ms — o custo de comparar d filhos por nível supera o ganho da menor altura. **d=4 é o ponto ótimo.**

C++ — Panorama Geral ($N = 10^6$, Aleatório)



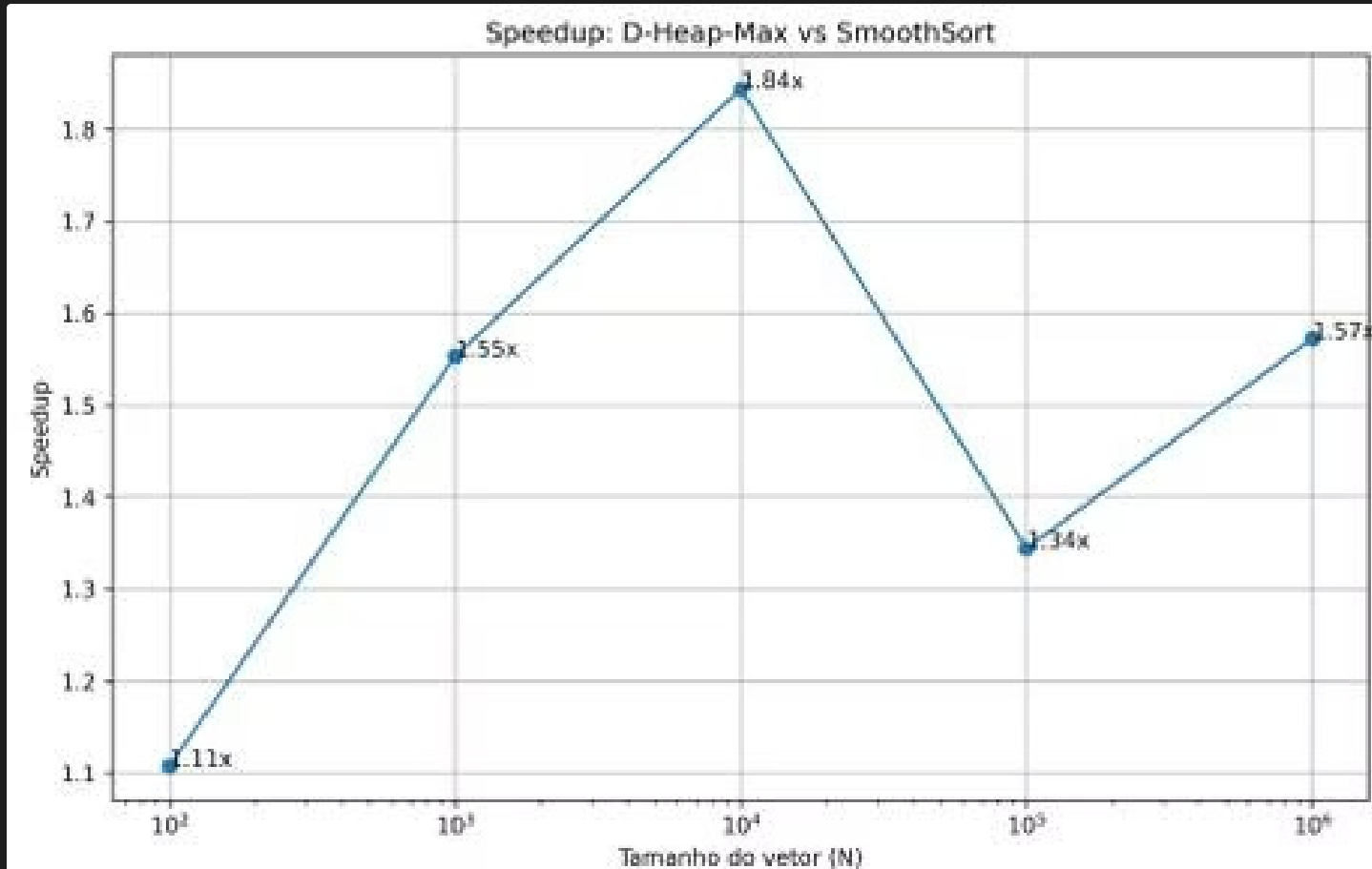
D-Heap-Max $d=4$ e D-Heap-Min $d=4$ registraram **~633 ms** — melhor desempenho entre todas as variantes. SmoothSort registrou ~996 ms em dados aleatórios.

C++ — Escalabilidade e Memória



Crescimento compatível com $O(n \log n)$. Memória cresce linearmente $O(n)$ — SmoothSort consome ligeiramente menos por não armazenar ponteiros adicionais.

C++ — Speedup: D-Heap-Max vs. SmoothSort



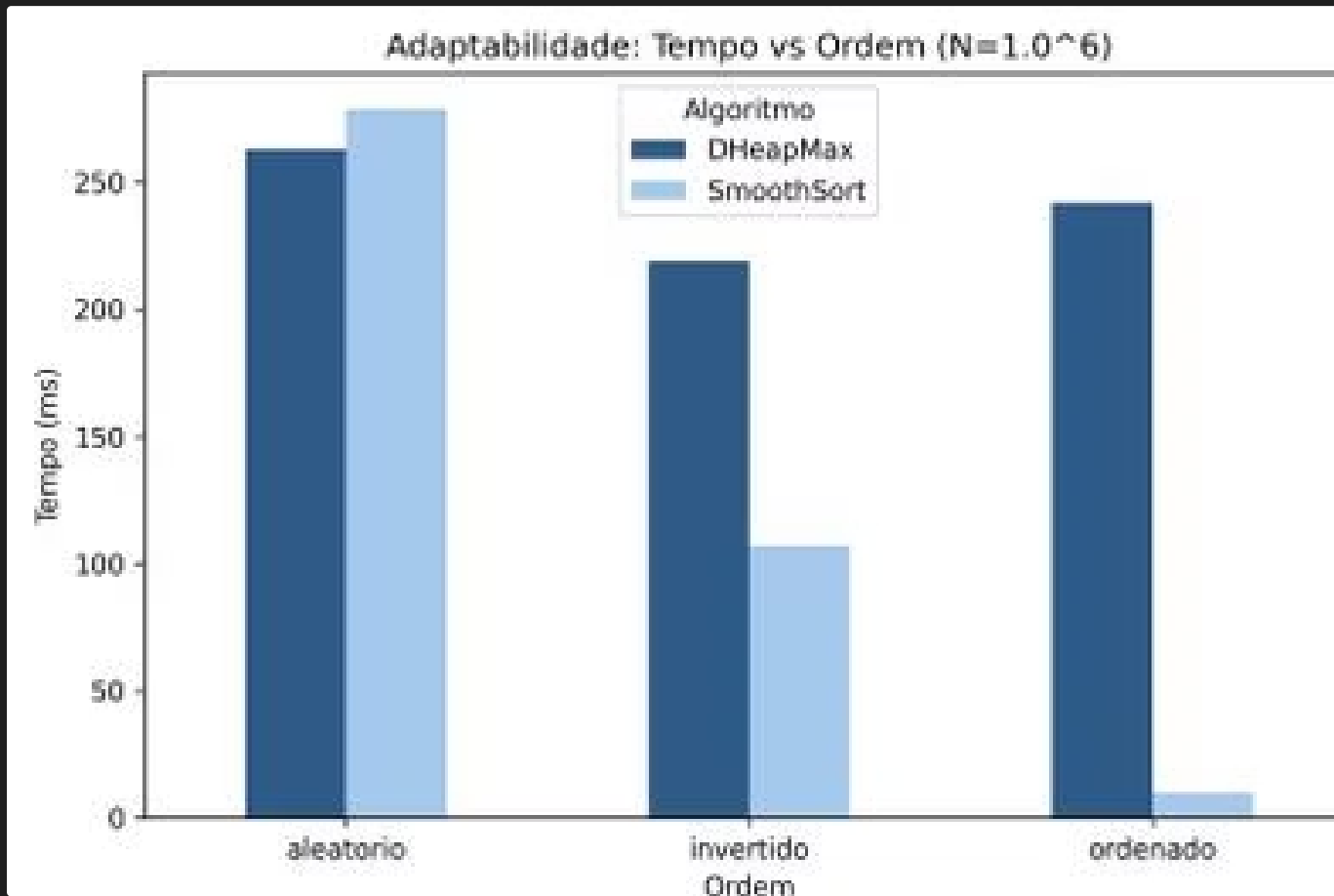
D-Heap-Max (d=4) foi **1,11x a 1,84x** mais rápido em dados aleatórios. Para $N=10^6$: speedup de **1,57x**. A vantagem se inverte completamente em dados ordenados.

CAPÍTULO 5

Resultados — Java

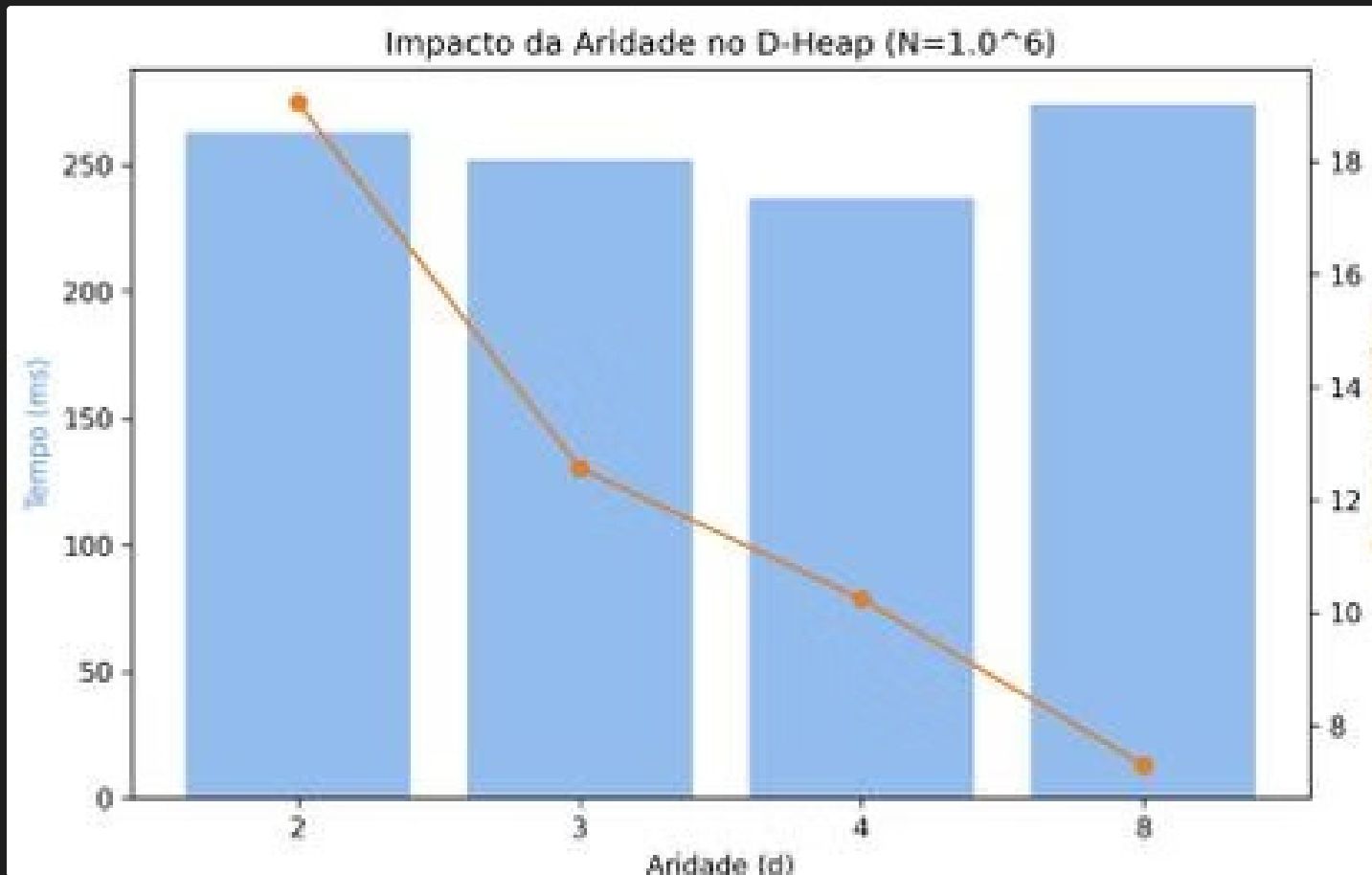
20 execuções por cenário. Métricas: tempo, swaps e memória.

Java — Adaptabilidade ($N = 10^6$)



Cenário	DHeapMax (ms)	SmoothSort (ms)
Aleatório	~263	~277
Invertido	~219	~108
Ordenado	~242	~10

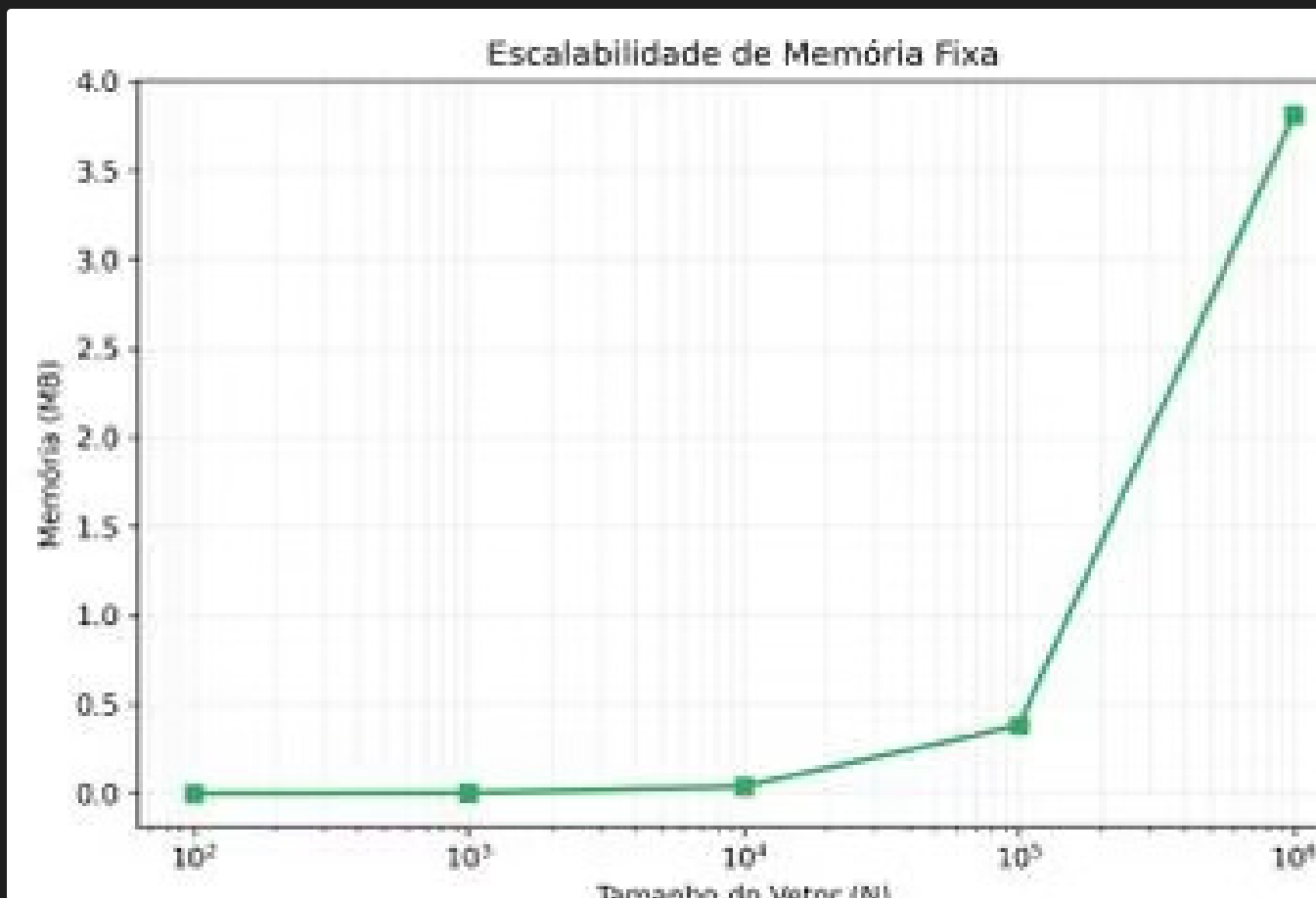
Java — Aridade e Trocas ($N = 10^6$)



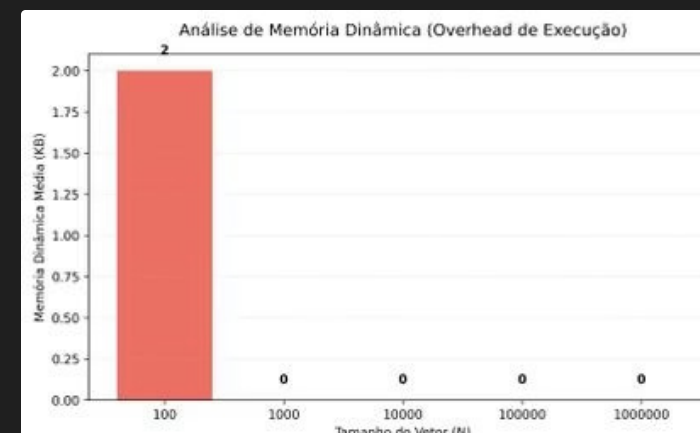
d	Tempo (ms)	Trocas (M)
2	263	19
3	242	13
4	219	10
8	272	7,5

Redução de trocas é consistente com o aumento de d , mas o tempo não acompanha linearmente — $d=4$ é o ponto ótimo.

Java — Memória Fixa e Dinâmica

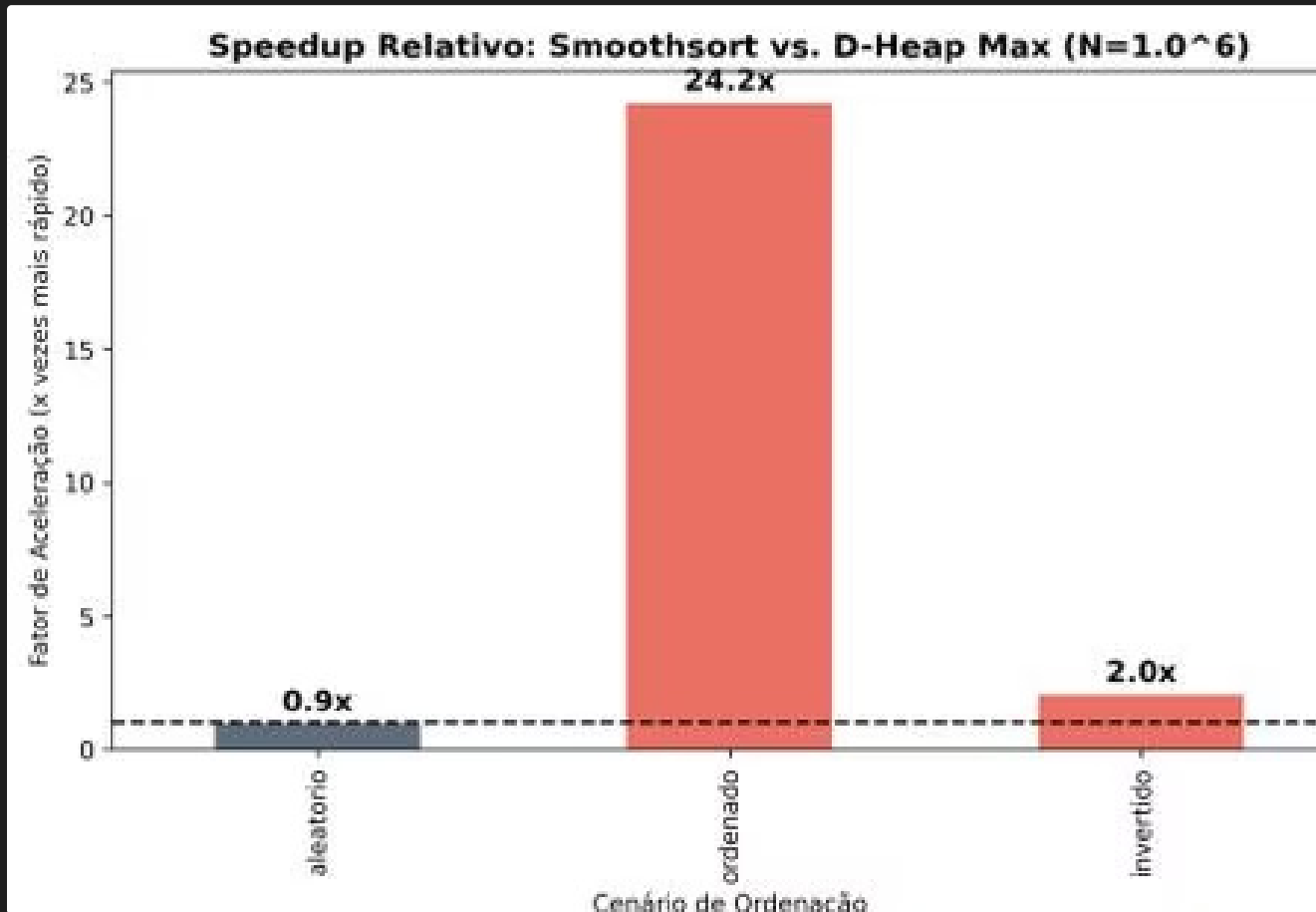


Memória fixa cresce linearmente: $N=10^6 \rightarrow \sim 3,8 \text{ MB}$ (coerente com $10^6 \times 4 \text{ bytes} = 4 \text{ MB}$).



Memória dinâmica: **0 KB** para $N \geq 10^3$.
Pico de 2 KB em $N=100$ é ruído da JVM — não representa consumo real do algoritmo.

Java — Speedup SmoothSort vs. DHeapMax



0,9x

Aleatório

SmoothSort ligeiramente mais lento

2x

Invertido

SmoothSort 2x mais rápido

24,2x

Ordenado

SmoothSort domina — confirma $O(n)$

CAPÍTULO 5

Resultados — Python e Go

Python — Comparação por Tipo de Entrada



Aleatório



Ordenado

⚠ Python usou Timsort nativo como proxy do SmoothSort — resultados refletem o ambiente, não o algoritmo original.

Python — Consumo de Memória



HeapSort Max/Min/D-ário: consumo praticamente constante. **SmoothSort (Timsort)** apresentou crescimento expressivo de memória — ~10.000 bytes para $N=10^6$ — reflexo da implementação híbrida nativa.

Go — SmoothSort em Diferentes Cenários



N	Ordenado (ms)	Invertido (ms)	Aleatório (ms)
10 ²	4,26	7,99	9,33
10 ³	46,0	62,2	90,5
10 ⁴	37,1	62,2	90,5

Entradas ordenadas consistentemente mais rápidas — comportamento adaptativo confirmado. Testes executados uma única vez por limitação de prazo.

CAPÍTULO 6

Análise Crítica

Comparação entre linguagens, limitações metodológicas e achados principais.

Comparação entre Linguagens — Tempos Absolutos

Linguagem	Algoritmo	N=10 ⁶ Aleatório	Observação
C	SmoothSort	~0,042 s (ordenado)	Melhor em dados ordenados
C++	D-Heap-Max d=4	~633 ms	Melhor em dados aleatórios
Java	D-Heap-Max d=4	~263 ms	JIT compensa overhead JVM
Python	SmoothSort (Timsort)	Superior aos demais	Timsort ≠ SmoothSort original

 Hardware distinto por integrante — comparação direta entre linguagens tem limitação metodológica relevante.

Achado Principal: Superioridade Adaptativa

24,2x

Java

Speedup SmoothSort vs. D-Heap-
Max em dados ordenados ($N=10^6$)

56ms

C++

SmoothSort em dados ordenados
vs. ~525 ms do D-Heap-Max

0,042s

C

SmoothSort em dados ordenados
— muito abaixo dos demais



Limitações Metodológicas

Hardware Heterogêneo

Cada integrante usou máquina diferente — comparação entre C, C++ e Java não é direta nem isenta

Python $\neq$ SmoothSort

Timsort nativo é um algoritmo híbrido altamente otimizado em C — não representa implementação fiel do SmoothSort original

Poucas Repetições em C/C++

3 repetições (C) e 5 (C++) vs. 20 (Java) — maior variância estatística nas linguagens compiladas

Go Limitado

Apenas uma execução por cenário e N máximo de 10^4 — resultados indicativos, não conclusivos

Dataset Real: MovieLens 25M



Houve um erro ao gerar esta imagem

Validação em Dados Reais

O arquivo `ratings.csv` contém milhões de avaliações de filmes, representando um cenário de processamento em larga escala.

- SmoothSort: menores tempos em entradas previamente ordenadas
- HeapSort (variantes): comportamento estável em consumo de memória
- Resultados reforçam aplicabilidade em pipelines de dados reais

Trade-offs: Quando Usar Cada Algoritmo?



Resumo Prático

- **SmoothSort:** logs, sensores, dados quase ordenados
- **D-Heap $d=4$:** dados aleatórios, sistemas de tempo real
- **HeapSort binário:** simplicidade e previsibilidade

❏ Nenhum algoritmo é universalmente superior — o perfil da entrada é determinante.

CAPÍTULO 7

Conclusões

Síntese dos Achados

SmoothSort é adaptativo de forma comprovada

Speedup de até 24,2x (Java) e ~11x (C++) em dados ordenados — confirma complexidade $O(n)$ no melhor caso experimentalmente.

D-Heap $d=4$ é o ponto ótimo

Equilíbrio entre altura da árvore e custo por nível — consistentemente superior a $d=2$, $d=3$ e $d=8$ nos experimentos.

Todos os algoritmos são genuinamente in-place

Memória dinâmica de 0 KB para $N \geq 10^3$ em Java — adequados para ambientes com restrições severas de memória.

Propostas de Trabalhos Futuros

Hardware Uniforme

Repetir experimentos em ambiente controlado único para comparação direta entre linguagens

SmoothSort Real em Python

Implementar o SmoothSort original em Python, sem usar Timsort nativo

Mais Aridades

Testar $d \in \{5, 6, 7\}$ para refinar o ponto ótimo da aridade

Dados Parcialmente Ordenados

Avaliar SmoothSort com graus variados de ordenação prévia (10%, 50%, 90%)

Referências Principais

Cormen et al. (2009)

Introduction to Algorithms, 3ª ed. MIT Press — base teórica do HeapSort e complexidade $O(n \log n)$

Johnson (1975)

"Priority queues with update and finding minimum spanning trees" — proposta original do heap d-ário

Dijkstra (1982)

"Smoothsort, an alternative for sorting in situ" — *Science of Computer Programming*, vol. 1, pp. 223–233

Tarjan (1983)

Data Structures and Network Algorithms — formalização do heap d-ário e análise de aridades ótimas

Código-fonte disponível em: github.com/alexcadamaso/Trabalho_Seminario_AEDS

Obrigado!

Alexandre Dâmaso · Julya Pires · Mariana Kaori Yano · Rita Marie

Engenharia de Computação — CEFET-MG, Divinópolis



alexadamaso@gmail.com

julyapires500@gmail.com

marisxzh@gmail.com

ritamariecajuru@gmail.com